

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	K. HEMASREE	Reg. No.:	192411156
Section / Batch:	SLOT C	Date:	23/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I, K. HEMASREE(192411156)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
-------------------	--------------------	--------------------

Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

AIM

To construct the **Minimum Spanning Tree (MST)** of a weighted graph using **Prim's and Kruskal's algorithms**, compare their edge-selection strategies, and analyze their suitability for dense and sparse graphs.

ALGORITHM

Prim's Algorithm

1. Start with any vertex.
2. Mark the selected vertex as visited.
3. Find the minimum-weight edge connecting a visited vertex to an unvisited vertex.
4. Add this edge to the MST.
5. Mark the newly selected vertex as visited.
6. Repeat Steps 3–5 until all vertices are included.
7. Calculate and display the total MST weight.

Kruskal's Algorithm

1. Store all graph edges with their weights.
2. Sort all edges in increasing order of weight.
3. Initialize each vertex as a separate set.
4. Select the smallest edge.
5. Check whether adding the edge creates a cycle using **Union-Find**.
6. If no cycle is formed, add the edge to the MST.
7. Repeat until $V - 1$ edges are selected.
8. Calculate and display the total MST weight.

Comparison

- **Prim's:** Grows the MST continuously from a selected vertex.
- **Kruskal's:** Selects the globally smallest available edges.
- **Dense graphs:** Prim's algorithm is generally more suitable.
- **Sparse graphs:** Kruskal's algorithm is generally effective.
- **Prim's complexity:** $O(E \log V)$ using a priority queue.
- **Kruskal's complexity:** $O(E \log E)$.

INPUT

Use the input format expected by the program:

Enter number of vertices: 5

Enter number of edges: 7

Enter edges (u v weight):

0 1 2

0 3 6

1 2 3

1 3 8

1 4 5

2 4 7

3 4 9

OUTPUT

--- MST Comparative Analysis ---

Prim's Algorithm:

Edges in MST:

0 - 1 : 2

1 - 2 : 3

1 - 4 : 5

0 - 3 : 6

Total Weight: 16

Kruskal's Algorithm:

Edges in MST:

0 - 1 : 2

1 - 2 : 3

1 - 4 : 5

0 - 3 : 6

Total Weight: 16

Comparison:

Prim's Algorithm: Suitable for dense graphs.

Kruskal's Algorithm: Suitable for sparse graphs.

Both algorithms produce the same minimum total weight.

Language: Python C C++ Java

QUESTIONS

Q1
Prim's vs Kruskal's Algorithm
(MST Construction)
100 marks

1/1 answered

Q1 Prim's vs Kruskal's Algorithm (MST Construction)

100 marks

Given a graph, construct MST using Prim's and Kruskal's methods. Compare edge selection strategies. Analyze efficiency in dense vs sparse graphs. Generate insights on MST algorithm suitability.

Your Code

```
1 import heapq
2
3 class Graph:
4     def __init__(self, vertices):
5         self.V = vertices
6         self.graph = [] # For Kruskal's: list of (weight, u, v)
7         self.adj = [[] for _ in range(vertices)] # For Prim's: adjacency list
8
9     def add_edge(self, u, v, w):
10        self.graph.append((w, u, v))
11        self.adj[u].append((v, w))
12        self.adj[v].append((u, w))
13
14    # Kruskal's Algorithm
15    def find(self, parent, i):
16        if parent[i] == i:
17            return i
18        return self.find(parent, parent[i])
19
20    def union(self, parent, rank, x, y):
21        root_x = self.find(parent, x)
22        root_y = self.find(parent, y)
23        if rank[root_x] < rank[root_y]:
24            parent[root_x] = root_y
25        elif rank[root_x] > rank[root_y]:
26            parent[root_y] = root_x
27        else:
28            parent[root_x] = root_y
29            rank[root_x] += 1
```

Input

stdin input

Output

--- MST Comparative Analysis ---
Enter number of vertices: Error:
EOF when reading a line. Please
ensure input matches the
required format.